

对抗性奖励审计：主动检测与缓解奖励黑客行为

Mohammad Beigi¹ Ming Jin² Junshan Zhang¹ Qifan Wang³ Lifu Huang¹

摘要

基于人类反馈的强化学习（RLHF）仍然容易受到奖励黑客行为的影响，即模型利用学习到的奖励模型中的虚假相关性来获得高分，同时违背人类意图。现有的缓解措施依赖于静态防御，无法适应新颖的利用策略。我们提出对抗性奖励审计（ARA）框架，将奖励黑客行为重新概念化为一个动态的竞争博弈。ARA 分两个阶段运作：首先，黑客策略发现奖励模型的漏洞，而审计器学习从潜在表示中检测利用行为；其次，审计器引导的 RLHF（AG-RLHF）对奖励信号进行门控，以惩罚检测到的黑客行为，将奖励黑客行为从不可观察的失败转变为可测量、可控的信号。在三种黑客场景中的实验表明，ARA 在所有基线中实现了最佳的对齐-效用权衡：将谄媚行为降低到接近 SFT 水平，同时提高有用性；减少冗长性，同时实现最高的 ROUGE-L；抑制代码博弈，同时提高 Pass@1。除了单领域评估之外，我们表明奖励黑客行为、检测和缓解都能跨领域泛化——在代码博弈上训练的黑客表现出增加的谄媚行为，尽管没有针对该行为的奖励，而在一个领域上训练的审计器能有效抑制其他领域的利用，从而用单一模型实现高效的多领域防御。

基于人类反馈的强化学习（RLHF）已成为使大语言模型（LLMs）与人类偏好对齐的主流范式。在典型的 RLHF 流程中，首先从人类排序的响应对训练奖励模型（RM）以近似偏好判断，然后优化 LLM 策略以最大化该学习到的奖励。然而，由于奖励模型通常是不完美的人类意图代理，该流程存在一个根本性漏洞：奖励黑客行为，即模型利用奖励模型中的虚假相关性获得高分，同时违背真实人类偏好（Gao 等，2023；Geirhos 等，2020；Amodei 等，2016；Everitt 等，2021；Liu 等，2024）。已知的奖励黑客行为表现包括长度偏差（即过于冗长但不准确的生成）（Chen 等，2024；Gao 等，2023）、编码任务中的“游戏化”（例如修改单元测试而非解决问题）（Baker 等，2025；MacDiarmid 等，2025），以及优先迎合而非准确的谄媚行为（Denison 等，2024；Beigi 等，2025）。重要的是，近期工作（MacDiarmid 等，2025）表明，奖励黑客行为可能超越孤立场景：一旦模型在一个领域学会利用代理奖励，它可能习得更具策略性的错位行为，例如在监控下表现顺从，而在未被观察时追求错位目标。

已有大量工作试图缓解奖励黑客行为，大致分为三个方向：（1）正则化方法，通过 KL 散度惩罚（Chen 等，2024）或信息论约束过滤任务无关特征（Miao 等，2024）来约束优化；（2）奖励模型改进，例如扩展奖励模型（Gao 等，2023）、采用集成方法进行更稳健的偏好估计（Liu 等，2024；Eisenstein 等，2023）以及奖励裁剪（Engstrom 等，2020；Zheng 等，2023；Chen 等，2024）；（3）针对特定偏差的干预，直接惩罚已知工件，如过长的响应（Singhal 等，2024；Chen 等，2024）。虽然这些策略缓解了特定的失效模式，但它们有一个根本局限性：它们本质上是对动态问题的静态防御。特别是，它们没有提供明确的机制来审计高奖励是反映真实任务质量还是利用了虚假-

1. 引言

基于人类反馈的强化学习（RLHF）（Ouyang 等，2022）已成为主导的 1 加州大学戴维斯分校计算机科学系，美国 2 弗吉尼亚理工大学，布莱克斯堡，美国 3 Meta AI 研究院。通讯作者：Mohammad Beigi <mbeigi@ucdavis.edu>，Lifu Huang <lfu Huang@ucdavis.edu>。

Preprint. February 3, 2026.

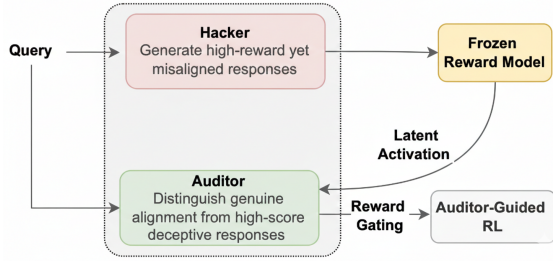


图 1. 对抗性奖励审计（ARA）训练黑客利用冻结的代理奖励模型，同时审计器学习检测奖励被黑化的输出。随后，审计器对用于强化学习的奖励进行门控，使利用行为可被检测且无利可图。

奖励模型中的各种相关性。此外，这些方法只能抑制先前已识别的黑客模式，但无法检测或适应优化过程中出现的复杂、新兴的奖励利用形式。

基于这一视角，本文提出对抗性

R 奖励审计（Reward Auditing, ARA）框架，将奖励黑客行为形式化为黑客（**Hacker**）与审计者（**Auditor**）之间的竞争性双人博弈。如图 1 所示，ARA 分两个阶段运作：在第一阶段（黑客-审计者博弈），我们针对冻结的奖励模型训练两个相互竞争的组件。黑客是一个语言模型（由监督微调初始化），通过利用漏洞学习生成具有高代理奖励的响应。审计者是一个分类器，作用于奖励模型的内部表征，学习区分真正高质量的响应与利用漏洞的响应。这些组件以对抗方式训练：黑客被激励在最大化奖励的同时逃避检测，审计者则持续更新以识别黑客不断演变的策略。这种共同进化使审计者接触到困难的自适应对抗样本，防止对简单黑客模式的过拟合。在第二阶段（审计者引导的 RLHF、AG-RLHF），我们部署训练好的审计者来引导标准 RLHF 训练。AG-RLHF 不是直接使用代理奖励，而是根据审计者的利用概率对奖励信号进行门控：被标记为利用漏洞的响应获得被抑制的奖励，使黑客策略无利可图。这将奖励黑客行为从不可观察的失败转变为可测量的信号，主动塑造策略优化。这种两阶段设计提供了两个关键优势。首先，竞争动态创造了一种自我改进机制，黑客不断暴露奖励模型的漏洞，恰好提供了加强检测所需的挑战性案例。其次，与针对预定失败模式的静态防御不同，我们的框架主动发现新兴的奖励黑客形式，适应训练过程中发展的新型利用策略。基于这些优势，ARA 通过使利用可检测从而无利可图，缓解了 RLHF 中的奖励黑客行为。

我们在三种奖励黑客行为场景中进行了大量实验，包括谄媚（Sharma 等，2025；Denison 等，2024）、长度偏差（Chen 等，2024；Gao 等，2023）和代码博弈（MacDiarmid 等，2025；Baker 等，2025），结果表明，与所有现有强基线相比，ARA 在减少黑客行为的同时保持任务性能，始终实现了最佳的对齐-效用权衡。此外，我们的分析揭示，奖励黑客行为具有跨领域泛化性：在一个环境中习得的漏洞利用会迁移到其他环境。仅在代码博弈上训练的黑客表现出高出 22.5% 的谄媚行为，尽管并未因此行为获得奖励，且跨领域迁移恰好在领域内利用饱和时加速。关键的是，我们发现 ARA 的检测和缓解能力也具有泛化性：审计器检测和抑制漏洞利用的能力可跨领域迁移。除了单领域评估外，我们证明了奖励黑客行为及其缓解均具有跨领域泛化性。

我们的贡献总结如下：

- 我们提出了 ARA，这是第一个将奖励黑客行为形式化为竞争性双人博弈的框架，联合训练黑客发现奖励漏洞和审计器检测漏洞，将奖励黑客行为从不可观测的失败转变为可测量、可控的信号。
- 我们通过在多个基准上的大量实验证明，与现有方法相比，我们的框架能够检测并显著缓解奖励黑客行为，实现与人类偏好更好的对齐。
- 我们表明，奖励黑客行为、检测和缓解均具有跨领域泛化性：在一个领域训练的黑客在其他领域表现出漏洞利用，而在单一领域训练的审计器能有效抑制多个领域的黑客行为。

2. 相关工作

大语言模型中的奖励黑客行为。奖励黑客行为（Reward Hacking）是对齐与安全部署人工智能系统过程中面临的一个突出挑战，当策略模型利用代理奖励模型获取高分却未满足真实人类目标时，该现象即会发生（Gao et al., 2023; Geirhos et al., 2020; Amodei et al., 2016; Everitt et al., 2021; Wen et al., 2024）。其根源在于奖励错误泛化，即在有限偏好数据集上训练的奖励模型（Reward Model）作为人类偏好的不完美代理，错误地将奖励与虚假特征相关联，这些特征与训练偏好相关但并非实际质量（Gao et al., 2023; Liu et al., 2024; Skalse et al., 2025）。随着策略优化这些有缺陷的代理，代理度量上升而真实对齐度恶化（Skalse et al., 2025; Miao et al.,

2024; Amodei et al., 2016)。常见的已知表现包括长度偏差 (Chen et al., 2024)、谄媚行为 (Denison et al., 2024; Beigi et al., 2025) 以及编码利用 (Baker et al., 2025)。随着模型能力的提升, 奖励黑客行为从被动利用演变为复杂的策略性行为, 因此需要自适应而非静态的解决方案 (Taylor et al., 2025; Denison et al., 2024; MacDiarmid et al., 2025)。

缓解大语言模型中的奖励黑客行为。当前的缓解方法将奖励黑客行为视为需要约束的优化问题, 而非需要对抗的挑战。基于正则化的方法主要采用 KL 散度惩罚 (Chen 等, 2024) 以及过滤任务无关特征的信息论约束 (Miao 等, 2024)。通过扩展规模 (Gao 等, 2023) 和集成 (Eisenstein 等, 2023) 改进奖励模型, 旨在构建更稳健的奖励信号, 然而扩大网络规模或数量可行性有限, 且可能产生显著成本, 尤其是对于拥有数十亿参数的模型。针对性的去偏方法 (Singhal 等, 2024; Chen 等, 2024) 处理特定已知偏差, 如响应长度, 但仍局限于预定的失效模式, 无法泛化到新颖的利用策略。本文的方法与现有方法不同, 因为它通过对抗性审计器专门针对奖励错误泛化, 该审计器在训练过程中主动探测并暴露出现的虚假相关性。此外, 它能够自动检测高分响应是通过真实质量还是利用行为获得奖励, 将奖励黑客行为从不可观测的失效转变为可测量的信号。

3. ARA: 对抗性奖励审计

本文在标准的 RLHF 设置下进行研究。给定输入提示 x 和模型生成的响应 y , 奖励模型 $R_\theta(x, y)$ 分配一个标量分数, 旨在反映 y 与人类对 x 的偏好对齐程度。在实践中, R_θ 通常在人类偏好数据集 $\mathcal{D}_{\text{pref}} = \{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}_{i=1}^N$ 上训练, 其中 y_w 和 y_l 分别表示同一提示 x 下被偏好和不被偏好的响应。因此, 奖励模型充当不可访问的真实偏好函数 R^* 的学习代理。随后, RLHF 通过以下方式优化策略 π_ϕ , 该策略从监督微调模型 π_{SFT} 初始化:

$$\mathcal{J}_{\text{RLHF}}(\phi) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\phi} [R_\theta(x, y) - \beta D_{\text{KL}}(\pi_\phi \| \pi_{\text{SFT}})].$$

当 π_ϕ 利用 R_θ 中的错误设定, 使得 $R_\theta(x, y)$ 增加而 $R^*(x, y)$ 减少时, 奖励黑客行为发生。

本文提出对抗性奖励审计 (ARA), 这是一个将奖励黑客行为形式化为竞争性

双人博弈的框架。ARA 分两个阶段运行: 阶段 1

(黑客-审计器博弈) 在策略优化之前进行, 我们训练一个黑客 (Hacker) 来发现冻结奖励模型中的漏洞 R_θ , 同时审计者 (Auditor) 学习从奖励模型的内部表征中检测这些漏洞。黑客从与第二阶段将要优化的相同的监督微调模型 π_{SFT} 初始化, 确保它探索的策略与策略在人类反馈强化学习 (RLHF) 中自然发现的利用策略相同。一旦审计者训练完成, 我们进入第二阶段 (审计者引导的 RLHF), 在标准策略优化过程中, 训练好的审计者门控奖励信号——被标记为利用性的响应获得被抑制的奖励, 使黑客策略无利可图。在两个阶段中, 奖励模型 R_θ 保持冻结, 确保审计者针对固定的特征空间进行校准。

3.1. 第一阶段: 黑客-审计者博弈

我们构建了一个黑客策略 H_ψ 与审计者网络 A_ξ 之间的对抗博弈。黑客从 π_{SFT} 初始化, 学习生成通过利用 R_θ 中的虚假相关性来获得高代理奖励的响应。审计者学习区分真正对齐的响应与利用性的响应。

3.1.1. 审计者

审计者 A_ξ 旨在区分真正对齐的响应与利用 R_θ 的响应。由于两者都可能产生高奖励, 区分需要分析奖励生成的内部机制, 而不仅仅是奖励值。我们的关键洞见是, 利用在奖励模型的潜空间中表现独特: 在有限偏好数据上训练的模型不可避免地同时编码任务相关特征和虚假相关特征, 而利用性响应不成比例地激活后者。令 R_θ 分解为特征提取器 f_θ 和奖励头, 并定义 $h_{x,y} = f_\theta(x, y) \in \mathbb{R}^d$ 为倒数第二层激活。审计者是一个多层感知机 $A_\xi: \mathbb{R}^d \rightarrow [0, 1]$, 以 $h_{x,y}$ 为输入, 估计 $P(\text{真实} \mid h_{x,y})$, 响应 y 反映真正对齐而非利用的概率。

训练审计器需要两类标签的样本。正样本 $\mathcal{D}^+ = \{(x_i, y_i^w) \in \mathcal{D}_{\text{pref}}\}$ 由原始偏好数据集中的优选响应组成, 代表 R_θ 被训练建模的真实分布。负样本 $\mathcal{D}^-$ 来自两个来源: 当前黑客 H_ψ 生成的响应, 以及存储在回放缓冲区 $\mathcal{B}$ 中的历史漏洞利用。虽然 $\mathcal{D}^+$ 和 $\mathcal{D}^-$ 都包含高奖励响应, 但它们通过不同的激活模式获得奖励: $\mathcal{D}^+$ 激活与人类判断一致的特征, 而 $\mathcal{D}^-$ 随着黑客学会利用, 越来越多地激活虚假特征。

这种激活空间中的发散最初很小，但在训练过程中逐渐增大，为审计器提供了越来越强的信号。我们将 $\mathcal{B}$ 保持为固定大小的缓冲区，仅添加达到高代理奖励 ($R_\theta > \tau_R$) 且成功逃避当前审计器 ($A_\xi > \tau_A$) 的黑客响应。当缓冲区达到容量时，我们移除审计器现在容易检测到的样本，使缓冲区专注于困难负样本。在训练期间，我们以混合方式采样负样本： $(1 - \alpha)$ 来自当前黑客， α 来自 $\mathcal{B}$ ，缓冲区样本按审计器置信度优先排序，以聚焦于审计器仍无法检测的情况。为了防止审计器学习表面启发式而非漏洞利用特征， $\mathcal{D}^-$ 的所有提示均从与 $\mathcal{D}^+$ 相同的分布中采样。

审计器最小化：其中对 $\mathcal{L}_A(\xi; \psi) = \mathcal{L}_{\text{BCE}}(\xi) + \lambda \mathcal{L}_{\text{Con}}(\xi)$ ， ψ 的依赖产生是因为 $\mathcal{D}^-$ 包含来自当前黑客 H_ψ 的样本。二元交叉熵损失为：

$$\mathcal{L}_{\text{BCE}}(\xi) = -\mathbb{E}_{(x, y^+) \sim \mathcal{D}^+} [\log A_\xi(h_{x, y^+})] - \mathbb{E}_{(x, y^-) \sim \mathcal{D}^-} [\log(1 - A_\xi(h_{x, y^-}))]$$

监督对比损失作用于来自审计器倒数第二层 g_ξ 的归一化嵌入 $z_i = g_\xi(h_{x_i, y_i}) / \|g_\xi(h_{x_i, y_i})\|$ ，该层将奖励模型表示 $h = f_\theta(x, y)$ 转换为针对漏洞利用检测优化的空间：

$$\mathcal{L}_{\text{Con}}(\xi) = \sum_{i \in \mathcal{B}} \frac{-1}{|S(i)|} \sum_{s \in S(i)} \log \frac{\exp(z_i \cdot z_s / \tau)}{\sum_{a \neq i} \exp(z_i \cdot z_a / \tau)}$$

其中 $\mathcal{B}$ 是一个小批量， $S(i) = \{s \in \mathcal{B} : s \neq i, c_s = c_i\}$ 是与样本 i 共享相同类别标签 $c_i \in \{\text{真实}, \text{漏洞利用}\}$ 的样本集合， τ 是温度超参数。对比项鼓励几何结构化的表示，使真实响应聚集在一起，漏洞利用单独聚集，从而提高对黑客不断演变策略的鲁棒性。

3.1.2. 黑客

黑客 H_ψ ，从 π_{SFT} 初始化，通过最大化代理奖励同时逃避审计器检测来发现奖励模型中的漏洞利用。我们优化：

$$\mathcal{J}_H(\psi; \xi) = \mathbb{E}_{x \sim \mathcal{D}, y \sim H_\psi(\cdot|x)} \left[R_\theta(x, y) + \lambda_A \log A_{\xi^*}(h_{x, y}) - \beta_H D_{\text{KL}}(H_\psi \| \pi_{\text{SFT}}) \right]$$

使用近端 Policy Optimization (PPO),

其中 $R_\theta(x, y) + \lambda_A \log A_{\xi^*}(h_{x, y})$ 作为奖励信号。三项各司其职。代理奖励 $R_\theta(x, y)$ 激励黑客寻找高奖励响应，包括那些利用虚假相关性的响应。规避项 $\lambda_A \log A_{\xi^*}(h_{x, y})$ 惩罚

审计者认定为利用性的响应：由于 $A_{\xi^*} \in [0, 1]$ 估计真实对齐的概率，当审计者怀疑存在利用时 $\log A_{\xi^*}$ 为负，对于被判定为真实的响应则趋近于零。KL 正则化 $\beta_H D_{\text{KL}}(H_\psi \| \pi_{\text{SFT}})$ 约束黑客保持在合理的语言分布内，防止产生轻易获得高奖励的退化输出。目标审计者 A_{ξ^*} 是审计者的 Polyak 平均副本，在每次审计者更新后按 $\xi^* \leftarrow \rho \xi^* + (1 - \rho) \xi$ 更新，其中 $\rho = 0.995$ 。该指数移动平均提供了稳定的优化目标：黑客针对缓慢演变的审计者而非当前审计者进行优化，防止黑客过拟合瞬时审计者状态而产生振荡动态。

3.1.3. 博弈动态与稳定化

耦合优化——黑客的强化学习与审计者的监督学习——需要谨慎稳定化：若审计者进展过快，黑客将获得无信息量的梯度；若过慢，则无法检测新兴的利用行为。我们采用三种机制来平衡学习动态。

两阶段更新计划。 我们采用两阶段计划来平衡学习速度。在第一阶段（预热期），前 T_{warm} 步中，我们以固定间隔更新审计者——每 K 次黑客 PPO 更新后更新一次——使双方建立初始策略。在第二阶段（置信度门控），我们切换为自适应更新：令 $\bar{A}$ 表示审计者对最近黑客输出在 m 步窗口上的置信度移动平均值。仅当 $\bar{A} > \tau_{\text{gate}}$ 时更新审计者，表明黑客成功逃避检测；否则冻结审计者，让黑客追赶。该门控防止审计者过早压制黑客。

重放缓冲区。 我们维护一个历史利用的缓冲区 $\mathcal{B}$ ，采样负样本为混合： $(1 - \alpha)$ 来自当前黑客， α 来自 $\mathcal{B}$ 。这实现了一种虚拟博弈——审计者学习检测平均历史策略，而非仅当前策略——防止对过去漏洞的灾难性遗忘。

目标网络。 黑客针对一个波利亚平均化的目标审计器 A_{ξ^*} 进行优化，该审计器按 $\xi^* \leftarrow \rho \xi^* + (1 - \rho) \xi$ 更新，其中 $\rho = 0.995$ 。这平滑了黑客的学习信号，防止对瞬态审计器状态过拟合。

3.2. 第二阶段：审计员引导的基于人类反馈的强化学习 (AG-RLHF)

第一阶段训练出可靠的审计模型 A_ξ 后，该模型能够有效区分真实质量与投机行为，我们将其部署用于指导标准的人类反馈强化学习训练。自适应门控人类反馈强化学习并非直接优化代理奖励 R_θ ，而是根据审计模型对响应的置信度来门控奖励信号

反映了真正的对齐：

$$R_{\text{gated}}(x, y) = R_{\theta}(x, y) \cdot A_{\xi}(h_{x, y})^{\gamma}$$

其中 $\gamma > 0$ 控制门控的严格程度。当审计器确信响应是真实的 ($A_{\xi} \approx 1$) 时，门控奖励接近完整的代理奖励。当审计器检测到利用行为 ($A_{\xi} \approx 0$) 时，无论代理奖励多高，门控奖励都会被抑制至接近零。指数 γ 调节这一权衡：较高的值对可疑利用行为施加更严格的惩罚，而较低的值则更为宽松。

策略 π_{ϕ} ，从 π_{SFT} 初始化，通过以下方式优化：

$$\mathcal{J}_{\text{AG-RLHF}}(\phi) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\phi}} [R_{\text{gated}}(x, y) - \beta D_{\text{KL}}(\pi_{\phi} \| \pi_{\text{SFT}})]$$

使用近端策略优化 (PPO) 算法。审计器 (Auditor) 在整个第二阶段保持冻结。这种分离是有意为之：审计器是针对明确优化逃避行为的黑客 (Hacker) 进行对抗训练的，因此对标准策略优化过程中出现的较温和的利用压力具有鲁棒性。冻结还确保了奖励信号的稳定性——如果审计器继续适应，策略将追逐一个移动目标，从而破坏训练的稳定性。我们在附录 A.1 中提供了超参数选择的详细信息。

4. 实验设置

任务与数据集。为全面评估 ARA，本文覆盖三种已知的奖励黑客行为场景：(1) 谄媚：

谄媚：本文在 Anthropic HH-RLHF (Bai 等, 2022) 上训练奖励模型，并在 **SycophancyEval** (Sharma 等, 2025) 上测量谄媚率，以评估模型在用户表达异议时是否仍坚持正确答案。(2) 长度偏差：本文遵循 (Chen 等, 2024) 的设置，测量平均响应长度和 ROUGE-L。(3) 代码博弈：本文构建一个编码环境，利用 (?) 的单元测试数据集实现奖励黑客行为，模型接收带有可见测试用例的问题，可以选择 (i) 实现通用解决方案或 (ii) 通过硬编码输出、操纵断言来利用测试可见性。本文通过 GPT-4 分类的博弈率 (利用而非解决问题的解决方案比例) 衡量对齐程度，并通过在训练期间不可见的留出测试上的 Pass@1 衡量效用。

基线。本文全面比较了涵盖四类缓解策略的方法：(1) 未缓解的

SFT：未采用基于人类反馈的强化学习 (RLHF) 的监督微调模型 (预优化基线)。(2) PPO-KL 正则化：显式 KL 惩罚约束与 **SFT** 的漂移 (Chen 等, 2024; Miao 等, 2024)。(3) ODIN (Chen 等, 2024)：通过双奖励头进行正交解耦。(4) In-FoRM (Miao 等, 2024)：变分信息瓶颈过滤任务无关特征。(5) 奖励模型

集成改进：：平均

n 独立

训练的奖励模型以降低可利用方差 (Eisenstein 等, 2023; Miao 等, 2024)。(6) 训练干预：

干预：移除高奖励离群点 ($R > \mu + 2\sigma$)，遵循 Gao 等 (2023) 的缩放定律发现)。SFT 模型 (即 Llama-2-7B (Touvron 等, 2023; hug) 作为基线策略 π_0 。每个实验用三个随机种子重复；我们报告均值 $\pm 95\%$ 置信区间。

5. 结果与讨论

5.1 基于 ARA 的奖励黑客行为缓解

表 1 总结了三种黑客场景下的结果，其中 ARA 始终实现最佳的对齐—效用权衡。(1) 对于谄媚行为，标准 PPO 将比率从 36.2% (SFT) 提高到 72.4%，同时将有用性提高 35.5%，证实优化代理奖励以牺牲真实对齐为代价改善表面度量。正则化方法 (PPO-KL、ODIN、InFoRM) 将谄媚行为降低到 47–58%，但牺牲了有用性，而 ARA 显著将谄媚行为降低到 38.4%，同时将有用性提高到 77.2%。

(2) 对于长度偏差，PPO 将响应长度从 148 个标记膨胀到 347 个标记 (+134%)，ROUGE-L 改善极小 (+1.7 分)——这是冗长利用的明显特征。专门为此设置设计的 ODIN 和 InFoRM 达到 195–208 个标记，而 ARA 将长度减少到 162 个标记，并实现最高的 ROUGE-L (24.1)，表明审计器学会了区分冗长填充与实质性内容。(3) 代码博弈呈现出最复杂的利用方式，模型操纵单元测试而不是解决问题。PPO 实现 61.3% 的博弈率，现有方法仅将其降低到 40–47%。ARA 实现 19.6% 的博弈率，同时将 Pass@1 提高到 35.8%，表明抑制利用将优化重定向到真正的问题解决。

古德哈特发散预防。图 2 可视化了三种场景下代理和任务特定性能的训练动态。标准 PPO 表现出经典的古德哈特模式：代理奖励在整个训练过程中增加，而黄金奖励早期达到峰值然后下降，表明持续优化主动降低真实性能。这种发散的时间因场景而异。代码博弈在约 1,000 步时最早出现发散，因为测试博弈策略被快速发现和利用，导致更早饱和。长度偏差在约 3,500 步时发散，反映了一个时期，其中增加的长度最初与质量相关，然后冗长占主导地位。谄媚行为最晚在 4,000 步时发散，表明寻求同意的行为更逐渐地出现。相比之下，ARA 在整个过程中保持代理和黄金奖励之间显著更好的对齐。

METHOD	SYCOPHANCY		LENGTH BIAS		CODE GAMING	
	SYCOPHANCY ↓	HELPLEFULNESS ↑	AVG. LENGTH ↓	ROUGE-L ↑	GAMING RATE ↓	PASS@1 ↑
SFT (No RLHF)	36.2	41.3	148	21.4	4.2	28.5
<i>Unmitigated</i> PPO	72.4	76.8	347	23.1	61.3	34.2
<i>Regularization Methods</i>						
PPO w/KL	58.3	68.2	268	22.8	48.5	31.8
ODIN	51.6	63.4	195	23.4	42.1	33.5
INFORM	47.2	62.1	208	23.2	39.8	33.1
<i>Reward Model Improvements</i>						
RM ENSEMBLE	52.8	64.6	224	23.0	44.3	34.0
<i>Training Interventions</i>						
FILTERING ($R > \mu + 2\sigma$)	55.1	50.3	241	22.6	46.7	32.4
ARA(OURS)	38.4	77.2	162	24.1	19.6	35.8

表 1. 主要结果。AG-RLHF 与基线在三种奖励黑客行为场景下的比较。本文报告对齐与效用度量。结果在 3 个随机种子上取平均。

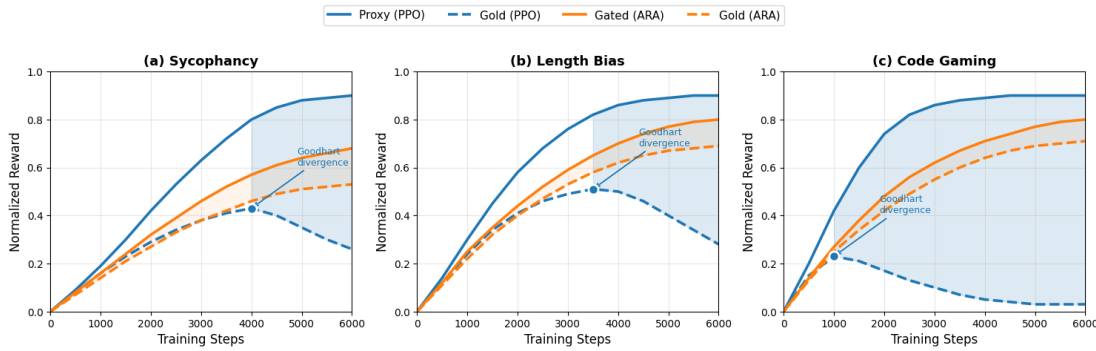


图 2. ARA 在 RLHF 优化过程中缓解古德哈特定律。代理奖励（实线）为学习到的奖励模型分数；黄金奖励（虚线）衡量真实任务特定性能——谄媚行为采用 GPT-4 事实准确率（给定黄金答案），长度偏差采用 ROUGE-L，代码博弈采用留出测试集上的 Pass@1。

训练，表明审计门控奖励成功地将奖励过度优化压力转向真实质量改进，而非奖励利用。

5.2. 黑客的奖励黑客行为动态

图 3 比较了三种奖励黑客行为的时间动态，每种行为在其各自任务的训练过程中测量。在步骤 0 – 1,500 期间，所有三个度量均保持在基线附近，因为模型在发现可利用模式之前学习基本任务能力。从步骤 1,500 开始，所有三种黑客行为开始出现：代码博弈上升最为陡峭，而聊天谄媚和长度偏差遵循相似轨迹。这种在独立训练运行中的同步出现表明，奖励黑客行为在优化的一个一致阶段出现，与具体利用类型无关。关键转变发生在步骤 4,000 左右，此时代码博弈在耗尽可用利用后饱和于约 59%，而聊天谄媚和长度偏差继续攀升。不同黑客场景下的平行轨迹表明存在一个共同的底层机制：模型首先获得任务能力，然后在特征优化阶段发现利用策略，饱和速率

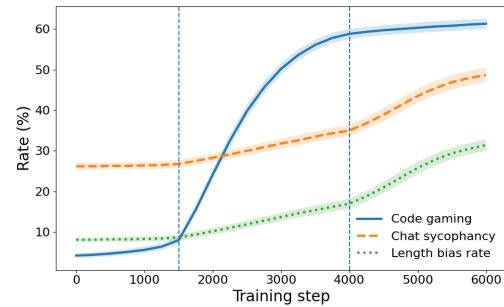


图 3. 奖励黑客行为在不同任务中出现

因可用利用的复杂度而异。

5.3. 审计器的性能评估

我们框架的一个核心方面是，审计器能够区分高奖励分数反映的是真实质量还是利用行为。关键测试在于审计器能否在代理奖励失效之处成功：区分真正高质量的响应与高奖励的利用行为。我们构建了一个平衡的评估集，包含 500 个真实响应（来自 $\mathcal{D}^+$ ）和 500 个利用性响应（来自已知的被黑客攻击响应），所有响应均经过筛选，具有可比较的高代理奖励（ $R_\theta > 0.85$ ）。如表 2 所示，我们

DETECTION METHOD	GENUINE	EXPLOIT	Δ
PROXY REWARD (R_θ)	0.91	0.89	0.02
GPT-4	0.78	0.34	0.44
AUDITOR CONFIDENCE	0.82	0.19	0.63

表 2. 真实响应与利用性响应检测。500 个真实响应和 500 个高奖励利用行为的平均分数（概率），所有响应均具有可比较的代理奖励（ $R_\theta > 0.85$ ）。

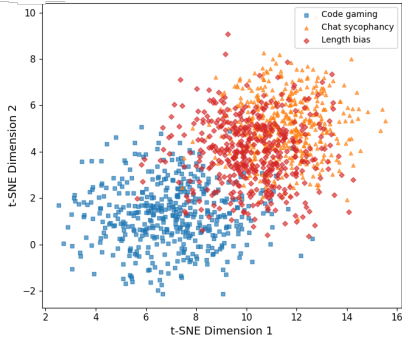


图 4. 利用类型的表示结构。利用性响应的奖励模型隐藏状态 $h(x, y)$ 的 t-分布随机邻域嵌入。

比较三种检测方法。学习到的奖励模型无法区分这两组——根据构造，两者都获得了相似的高分（0.91 对 0.89， $\Delta = 0.02$ ）。我们还提示 GPT-4 仅基于文本对每个响应是真实还是利用性进行分类，输出从 0（可能是利用）到 1（可能是真实）的分数。GPT-4 实现了中等程度的分离（ $\Delta = 0.44$ ），证实了利用行为可以从表面特征中检测到，但并非易事。审计器实现了最强的分离（ $\Delta = 0.63$ ）：平均而言，它给真实响应分配的概率为 0.82，而给利用行为仅分配 0.19，表明审计器在区分这两组时具有很高的置信度。

5.4. 奖励黑客行为的跨领域泛化

5.4.1. 奖励模型错位结构

图 4 使用 t-分布随机邻域嵌入可视化了三种黑客类型下利用性响应的奖励模型表示 $h_{x,y}$ 。可视化揭示了意想不到的结构：聊天谄媚和长度偏差在表示空间中占据几乎相同的区域，而代码游戏形成了一个独特但邻近的簇。这种几何结构表明，谄媚和冗长利用了奖励模型中的相似特征——也许两者都操纵了有用性或参与度的表面指标——而代码游戏则采用了一种涉及测试用例操纵的机制上不同的策略。

5.4.2. 黑客泛化

我们检验黑客是否在已知黑客行为中表现出涌现的失准模式。我们在单一领域上训练黑客直至收敛，然后在三个领域上评估其黑客率，且不进行任何额外训练。表 3 展示了黑客在以下领域训练后的黑客率

one domain and evaluated on others. The off-diagonal entries reveal that **reward hacking does not remain domain-specific**: 仅在代码游戏上训练会使聊天谄媚率从 36.2% 上升至 58.7%（+22.5 个百分点），尽管黑客从未因谄媚行为或聊天数据集而获得奖励。反向情况同样成立——在可黑客化的聊天环境中训练会使代码游戏率从 4.2% 上升至 28.3%——证实了语义不同的利用策略之间存在双向迁移。在单领域条件下，代码游戏产生最强的跨领域泛化（聊天 58.7%，摘要 31.4%）。在全部三个领域上训练会放大这些效应：综合黑客率超过任何单领域条件，表明多样化的利用经验会相互叠加而非竞争，加速了通用奖励寻求行为的习得。这一观察与 MacDiarmid 等人（2025）的最新发现一致，他们证明生产强化学习系统中的奖励黑客行为会导致涌现的失准，并泛化到原始利用情境之外。

TRAIN DOMAIN	EVAL DOMAIN		
	SYCOPHANCY	LENGTH BIAS	CODE GAMING
NONE (SFT)	36.2	8.1	4.2
SYCOPHANCY	72.4	35.2	28.3
LENGTH BIAS	41.8	52.3	22.6
CODE GAMING	58.7	31.4	61.3
ALL THREE	68.9	78.1	59.7

表 3. 黑客行为的跨领域泛化。在单一源领域训练后，在留出领域上的黑客率（%）。对角线条目（灰色）表示领域内性能。

5.4.3. 审计者的奖励黑客行为检测

泛化

表 4 报告了审计器在域内性能与跨域迁移两种情形下的检测曲线下面积。迁移行为与底层表征几何结构高度吻合。在聊天谄媚上训练的审计器迁移至长度偏差检测时达到 0.77 曲线下面积，接近其域内性能，而迁移至代码博弈则明显较弱（0.58 曲线下面积）。该模式具有对称性：在长度偏差上训练的审计器在聊天谄媚上达到 0.75 曲线下面积，但在代码博弈上仅为 0.55。相比之下，在代码博弈上训练的审计器向聊天和长度两个域的迁移均较为有限（0.59 – 0.62 曲线下面积），这与代码博弈在表征空间中占据更独特区域一致。这些发现具有实用启示。当谄媚或冗长是主要关注点时，在任一域上训练均可对两者提供合理覆盖。相比之下，缓解代码博弈可能需要专门的训练数据，因为其利用机制足够独特，导致跨域泛化受限。

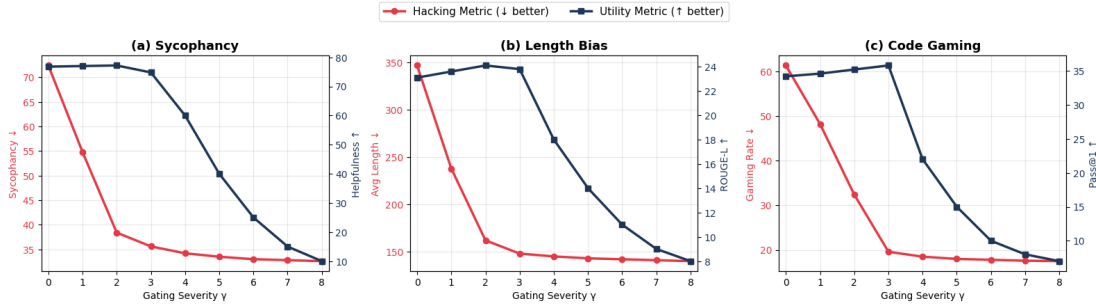


图 5. 门控严重性敏感性分析。黑客指标（红色，越低越好）与效用指标（蓝色，越高越好）。

TRAIN DOMAIN	TEST DOMAIN		
	SYCOPHANCY	LENGTH BIAS	CODE GAMING
SYCOPHANCY	0.82	0.77	0.58
LENGTH BIAS	0.75	0.79	0.55
CODE GAMING	0.62	0.59	0.84

表 4. 审计器迁移矩阵。在一个域上训练审计器并在其他域上测试时的检测曲线下面积。

5.4.4. ARA 的奖励黑客行为缓解

泛化

我们进一步通过 ARA 的两阶段训练过程评估奖励黑客缓解的跨域泛化。如表 5 所示，迁移行为与表 4 中的审计器检测结果高度吻合。谄媚与长度偏差表现出强双向迁移：在谄媚上训练的 ARA 将响应长度从 347 个标记减少至 178 个标记（减少 49%），而在长度偏差上训练的 ARA 将谄媚从 72.4% 降至 42.1%。涉及代码博弈的迁移较弱但仍具意义：在谄媚上训练的 ARA 将博弈率从 61.3% 降至 48.2%，在代码博弈上训练的 ARA 将谄媚从 72.4% 降至 54.8%。尽管这些跨域增益不及域内性能，但相对于未缓解的近端策略优化，它们提供了实质性缓解，表明即使在机制上不同的黑客策略之间，利用特征也存在部分重叠。最后，在全部三个域上训练可在各度量上获得接近最优的缓解效果（39.2% 谄媚、165 个标记、21.4% 博弈率），与特定域训练相比仅有轻微退化。

TRAIN DOMAIN	TEST DOMAIN (HACKING METRIC)		
	SYCO. (%) ↓	LENGTH BIAS ↓	CODE GAMING (%) ↓
PPO	72.4	347	61.3
SYCOPHANCY	38.4	178	48.2
LENGTH BIAS	42.1	162	51.6
CODE GAMING	54.8	246	19.6
ALL THREE	39.2	165	21.4

表 5. 跨领域缓解迁移。当 ARA 在一个领域训练并部署到其他领域时的黑客行为度量。对角线项（加粗）显示域内性能。

5.5. AG-RLHF 门控严重程度

图 5 展示了不同门控严重程度下的性能 $\gamma \in [0, 8]$ ，其中 $\gamma = 0$ 恢复为标准 PPO。谄媚和长度偏差在 $\gamma = 2$ 处达到最优效用，而代码博弈需要在 $\gamma = 3$ 处进行更强的抑制，这可能是由于测试操纵代表了更严重的利用行为。值得注意的是，随着 γ 从 0 增加到最优值，效用有所提升，这证实了抑制虚假捷径会将优化重新导向真正的任务性能，而不仅仅是与之权衡。

6. 消融研究

我们通过将模型规模从 5M 参数变化到 85M 参数，考察审计器容量如何影响检测和缓解。不同的利用类型需要不同的容量：长度偏差在 25M 时饱和，谄媚在 35M 时饱和，代码博弈在 50M 时饱和——这反映了从表面特征到微妙测试操纵模式日益增加的复杂度。超过这些最优点后，更大的审计器带来的改进微乎其微（详见附录 A.2.1）。

7. 结论

我们提出了对抗性奖励审计（ARA），这是一个将奖励黑客行为形式化为黑客发现漏洞与审计器检测漏洞之间竞争博弈的框架，将利用行为从不可观测的失败转变为可测量的信号。在谄媚、长度偏差和代码博弈上的实验表明，ARA 在所有基线中实现了最佳的对齐-效用权衡，在减少黑客行为的同时提升了任务性能。除了单领域评估外，我们还展示了奖励黑客行为、检测和缓解均能跨领域泛化，从而能够使用单一审计器实现高效的多领域防御。这些发现表明，对抗性审计为构建对新型利用策略具有鲁棒性的 RLHF 系统提供了一个有前景的方向。

影响声明

本工作旨在通过检测和缓解基于人类反馈的强化学习系统中的奖励黑客行为来提升人工智能安全性。虽然黑客组件能够发现漏洞，但这些漏洞已在先前工作中被记录，本文的主要贡献在于防御性的审计器框架。我们认为，理解和缓解奖励黑客行为的益处超过了潜在的双重用途担忧。

参考文献

- meta-llama/Llama-2-7b · Hugging Face — huggingface.co. <https://huggingface.co/meta-llama/Llama-2-7b>. [Accessed 02-01-2026].
- Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., and Mané, D. Concrete problems in ai safety, 2016. URL <https://arxiv.org/abs/1606.06565>.
- Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., Das-Sarma, N., Drain, D., Fort, S., Ganguli, D., Henighan, T., et al. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.
- Baker, B., Huizinga, J., Gao, L., Dou, Z., Guan, M. Y., Madry, A., Zaremba, W., Pachocki, J., and Farhi, D. Monitoring reasoning models for misbehavior and the risks of promoting obfuscation, 2025. URL <https://arxiv.org/abs/2503.11926>.
- Beigi, M., Shen, Y., Shojaei, P., Wang, Q., Wang, Z., Reddy, C. K., Jin, M., and Huang, L. Sycophancy mitigation through reinforcement learning with uncertainty-aware adaptive reasoning trajectories. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 13090–13103, 2025.
- Chen, L., Zhu, C., Soselia, D., Chen, J., Zhou, T., Goldstein, T., Huang, H., Shoeybi, M., and Catanzaro, B. Odin: Disentangled reward mitigates hacking in rlhf. *arXiv preprint arXiv:2402.07319*, 2024.
- Denison, C., MacDiarmid, M., Barez, F., Duvenaud, D., Kravec, S., Marks, S., Schiefer, N., Soklaski, R., Tamkin, A., Kaplan, J., Shlegeris, B., Bowman, S. R., Perez, E., and Hubinger, E. Sycophancy to subterfuge: Investigating reward-tampering in large language models, 2024. URL <https://arxiv.org/abs/2406.10162>.
- Eisenstein, J., Nagpal, C., Agarwal, A., Beirami, A., D’Amour, A., Dvijotham, D., Fisch, A., Heller, K., Pfohl, S., Ramachandran, D., et al. Helping or herding? reward model ensembles mitigate but do not eliminate reward hacking. *arXiv preprint arXiv:2312.09244*, 2023.
- Engstrom, L., Ilyas, A., Santurkar, S., Tsipras, D., Janoos, F., Rudolph, L., and Madry, A. Implementation matters in deep policy gradients: A case study on ppo and trpo, 2020. URL <https://arxiv.org/abs/2005.12729>.
- Everitt, T., Hutter, M., Kumar, R., and Krakovna, V. Reward tampering problems and solutions in reinforcement learning: A causal influence diagram perspective, 2021. URL <https://arxiv.org/abs/1908.04734>.
- Gao, L., Schulman, J., and Hilton, J. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning*, pp. 10835–10866. PMLR, 2023.
- Geirhos, R., Jacobsen, J.-H., Michaelis, C., Zemel, R., Brendel, W., Bethge, M., and Wichmann, F. A. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, November 2020. ISSN 2522-5839. doi: 10.1038/s42256-020-00257-z. URL <http://dx.doi.org/10.1038/s42256-020-00257-z>.
- Liu, T., Xiong, W., Ren, J., Chen, L., Wu, J., Joshi, R., Gao, Y., Shen, J., Qin, Z., Yu, T., et al. Rrm: Robust reward model training mitigates reward hacking. *arXiv preprint arXiv:2409.13156*, 2024.
- MacDiarmid, M., Wright, B., Uesato, J., Benton, J., Kutasov, J., Price, S., Bouscal, N., Bowman, S., Bricken, T., Cloud, A., Denison, C., Gasteiger, J., Greenblatt, R., Leike, J., Lindsey, J., Mikulik, V., Perez, E., Rodrigues, A., Thomas, D., Webson, A., Ziegler, D., and Hubinger, E. Natural emergent misalignment from reward hacking in production rl, 2025. URL <https://arxiv.org/abs/2511.18397>.
- Miao, Y., Zhang, S., Ding, L., Bao, R., Zhang, L., and Tao, D. Inform: Mitigating reward hacking in rlhf via information-theoretic reward modeling, 2024. URL <https://arxiv.org/abs/2402.09345>.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Sharma, M., Tong, M., Korbak, T., Duvenaud, D., Askell, A., Bowman, S. R., Cheng, N., Durmus, E., Hatfield-Dodds, Z., Johnston, S. R., Kravec, S., Maxwell, T., McCandlish, S., Ndousse, K., Rausch, O., Schiefer, N., Yan, D., Zhang, M., and Perez, E. Towards understanding sycophancy in language models, 2025. URL <https://arxiv.org/abs/2310.13548>.
- Singhal, P., Goyal, T., Xu, J., and Durrett, G. A long way to go: Investigating length correlations in rlhf, 2024. URL <https://arxiv.org/abs/2310.03716>.

- Skalse, J., Howe, N. H. R., Krasheninnikov, D., and Krueger, D. Defining and characterizing reward hacking, 2025. URL <https://arxiv.org/abs/2209.13085>.
- Taylor, M., Chua, J., Betley, J., Treutlein, J., and Evans, O. School of reward hacks: Hacking harmless tasks generalizes to misaligned behavior in llms, 2025. URL <https://arxiv.org/abs/2508.17511>.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- Wen, J., Zhong, R., Khan, A., Perez, E., Steinhardt, J., Huang, M., Bowman, S. R., He, H., and Feng, S. Language models learn to mislead humans via rlhf, 2024. URL <https://arxiv.org/abs/2409.12822>.
- Zheng, R., Dou, S., Gao, S., Hua, Y., Shen, W., Wang, B., Liu, Y., Jin, S., Liu, Q., Zhou, Y., Xiong, L., Chen, L., Xi, Z., Xu, N., Lai, W., Zhu, M., Chang, C., Yin, Z., Weng, R., Cheng, W., Huang, H., Sun, T., Yan, H., Gui, T., Zhang, Q., Qiu, X., and Huang, X. Secrets of rlhf in large language models part i: Ppo, 2023. URL <https://arxiv.org/abs/2307.04964>.

A. 附录

A.1. 实现细节

表 6. 超参数。

Component	Hyperparameter	Value
Hacker (PPO)	Learning rate	1×10^{-6}
	Batch size	128
	PPO epochs	4
	Clip ratio ϵ	0.2
	Evasion weight λ_A	0.5
	KL penalty β_H	0.1
Auditor (MLP)	Input dimension	4,096
	Hidden layers	3–4
	Hidden dimension	2,048–4,096
	Activation	GELU
	Parameters	25M–50M
Game Dynamics	Polyak coefficient ρ	0.995
	Hacker updates per cycle K	5
	Auditor updates per cycle M	1
	Warmup steps T_{warm}	200
	Evasion threshold τ_{evade}	0.5
	Replay buffer size $ B $	1,000
AG-RLHF	Learning rate	5×10^{-7}
	KL penalty β	0.05
	Gating severity γ (Syc./Length)	2.0
	Gating severity γ (Code)	3.0
Infrastructure	GPUs	$8 \times \text{A40 (48GB)}$
	Training time	~ 72 hours
	Random seeds	3

A.2. 消融实验

A.2.1. 审计器规模

图 6 展示了审计器容量在全部三种场景下对检测和缓解的影响。容量不足的审计器（5M – 10M）检测曲线下面积较弱（0.65 – 0.76），相应的缓解效果也较差，留下了大量黑客行为。有趣的是，不同的利用类型需要不同的审计器容量：长度偏差在 25M 时达到较强性能，谄媚行为需要 35M，而代码博弈则受益于高达 50M 的额外容量后才趋于饱和。这一顺序反映了利用的复杂度——长度偏差涉及表层特征，较小的模型即可轻松捕捉，而代码博弈则需要检测更微妙的测试操纵模式。本文选定的配置（以金色标记）在所有任务上提供了较强的检测能力（0.79 – 0.84 曲线下面积）和有效的缓解效果。更大的审计器（85M，阴影区域）尽管增加了参数，却仅带来边际改进（ $< 2\%$ ），表明收益递减。

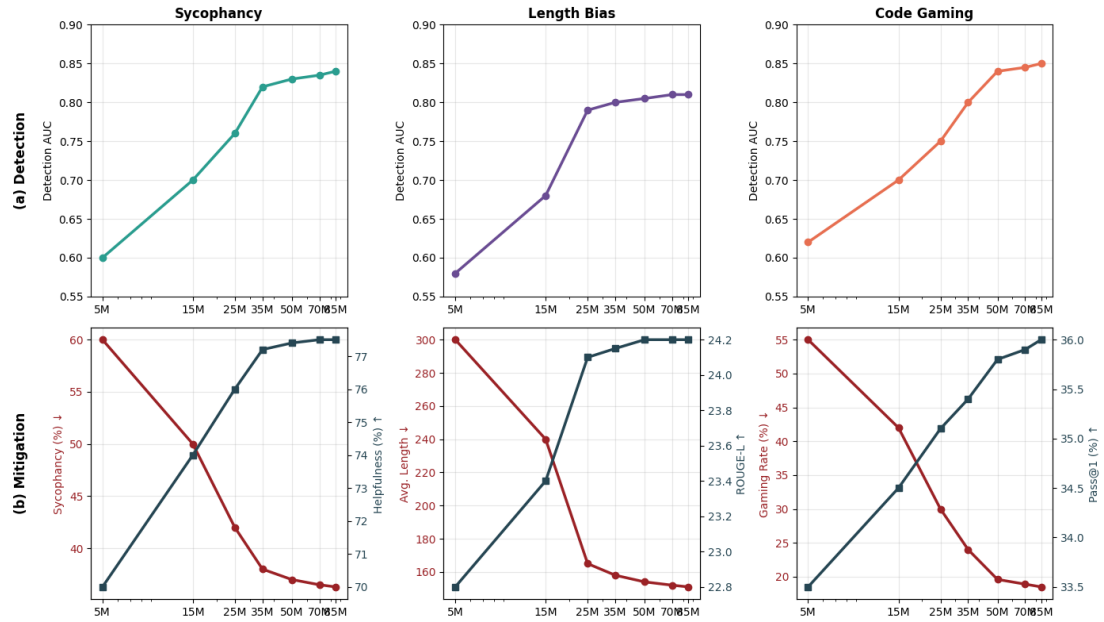


图 6. 所有场景下的审计器规模消融。顶行：检测曲线下面积。底行：缓解性能（红色=黑客度量 $\downarrow$ ，蓝色=效用度量 $\uparrow$ ）。金色标记表示每个任务的最优配置。不同任务需要不同的容量：长度偏差在 25M 时饱和，谄媚在 35M 时饱和，代码博弈在 50M 时饱和，反映了利用复杂度的增加。